

Programming

Lecture 03 — Decisions

Input, output, and programs that react to what they are given



Dr. Ramon Winterhalder

Human-Centered Artificial Intelligence · A.Y. 2026/27



UNIVERSITÀ
DEGLI STUDI
DI MILANO



Where we left off

Lecture 02, in four lines

A variable is a name attached to a value

and = attaches it. Right-hand side first, always.

Four types

int, float, str, bool — the type decides what the operators do.

Convert on purpose

int(), float(), str() — and int() chops rather than rounds.

Plan in comments

then fill the code in underneath.

Still missing: every value was written into the program, and every program did the same thing every time. Today it asks — and then decides.

Two promises come due today: the receipt that would not line up (sheet 1), and the shopping total we could not format (sheet 2).

Correction: how this course is assessed

Earlier I said the exam is pass/fail. That was wrong.

This course: graded out of 30

Written exam, closed book, voto in trentesimi.

Pass/fail: the Programming Lab

Pass/fail applies to the Lab, not to this course.

What is assessed

Correctness of your solutions

Understanding of programming concepts

Clarity of your algorithmic reasoning

Readable, working Python code

Exam format *the mock exam in lecture 12 has the same form*

1 True / false

10 statements on concepts

2 Multiple choice

12 short code fragments: what is the output?

3 Write a function

one open question

4 Write a program

one open question

By the end of today you can...

- read a number from the user, and explain why `int()` is always needed;
- print a formatted, aligned table with f-strings;
- write a condition that is either `True` or `False`, and combine conditions;
- choose between `if`, `elif` and `else` — and say why order matters;
- find the three bugs that live in every beginner conditional.

From today your programs can do different things on different runs. That is a much larger step than it looks.

A

Talking to the user

input · print · f-strings · formatting

Asking a question

```
name = input("What is your name? ")  
print("Hello,", name)
```

What happens

- Python prints the prompt and then waits.
- Nothing else runs until you press Enter.
- Whatever you typed becomes the value of name.

The prompt is just an argument to `input()`. Leave it out and the program waits in silence, which looks exactly like a crash — always give a prompt, and end it with a space.

input() always gives you text

The single most common bug of the week

```
age = input("Your age: ")      # you type 24
print(age + 1)
```

TypeError: can only concatenate str (not "int") to str

Even though you typed digits, age is the text "24" — not the number 24.

```
age = int(input("Your age: "))  # read it, then convert it
print(age + 1)                 # 25
```

Read the line from the inside out: input() first, then int() around it.

When the user types something else

```
>>> age = int(input("Your age: "))  
Your age: twenty  
ValueError: invalid literal for int() with base 10: 'twenty'
```

- The conversion failed, not the reading. `input()` did its job.
- This is the `ValueError` from last week, arriving from a new direction.
- For now we assume the user types what we asked for.
- Handling it properly needs `try / except` — lecture 09.

Worth knowing now: a program that trusts its input is fine for an exercise and dangerous everywhere else.

print(), more carefully

```
print("a", "b", "c")           # ->  a b c
print("a", "b", sep="")        # ->  abc
print("a", "b", sep=" | ")     # ->  a | b

print("no newline", end="")    # ->  stays on the line
print("a", end=" -> ")
print("b")                     # ->  a -> b
```

- sep decides what goes BETWEEN the arguments. Default: one space.
- end decides what goes AFTER the last one. Default: a newline.
- Both are keyword arguments — the name is part of the call.

f-strings

The one piece of syntax worth memorizing today

```
name = "Mirea"  
marks = 28  
  
print(f"{name} scored {marks} out of 30")
```

Mirea scored 28 out of 30

```
print(f"{marks} out of 30 is {marks / 30 * 100:.1f}%")
```

No more commas, no more str(), no more joining with +. Anything Python can compute goes inside the braces.

f

goes before the opening quote

{ }

anything inside is evaluated

{marks}

the value is dropped in, converted to text for you

Formatting inside the braces

Everything after the colon

spec	means	example	result
:.2f	two decimal places	f"{3.14159:.2f}"	3.14
:>8	right-aligned, width 8	f"{42:>8}"	" 42"
:<8	left-aligned, width 8	f"{'ab':<8}"	"ab "
:^8	centred, width 8	f"{'ab':^8}"	" ab "
:>8.2f	both together	f"{3.14159:>8.2f}"	" 3.14"
:,	thousands separator	f"{1234567:,}"	1,234,567

A width pads to at least that many characters, it never truncates. Numbers default to right, text to left — the arrow only matters when you want the other one.

You do not need to memorize these. You need to know they exist, and where to look them up.

Worked example

The receipt that would not line up

Sheet 1, spaced by hand

```
print("Coffee      x 3 =", 3 * 1.50)
print("Cornetto   x 2 =", 2 * 1.50)
```

```
Coffee      x 3 = 4.5
Cornetto    x 2 = 3.0
```

Fragile: change a name and the columns break. And 4.5 is not a price.

Today, with f-strings

```
print(f"{name:<10}{qty:>3} x {price:>6.2f}"
      f" = {qty * price:>7.2f}")
```

```
Coffee      3 x   1.50 =   4.50
Cornetto    2 x   1.50 =   3.00
Total                               7.50
```

The widths hold the columns, whatever the contents.

Build it in the notebook one column at a time, so the alignment appears as you go.

Exercises — Block A

Sheet 03, part A · 40 minutes

- Predict the output of eight f-strings.
- Fix four programs that forgot `int()` around `input()`.
- Convert three `print()` calls with commas into f-strings.
- Write a program that asks for a name and a year of birth, and prints an age.
- Print a small table that stays aligned when you change the values.

The last one is the real exercise. Everything before it is vocabulary.



Making decisions

Booleans · comparisons · if, elif, else

A question with a yes/no answer

```
>>> 7 > 3
True
```

```
>>> 7 == 3
False
```

```
>>> "a" < "b"
True
```

```
>>> 7 != 3
True
```

```
>>> 7 >= 7
True
```

```
>>> "Yes" == "yes"
False
```

operator	means	operator	means
==	equal to	!=	not equal to
<	less than	<=	less than or equal
>	greater than	>=	greater than or equal

= assigns a value. == asks a question. Mixing them up is a syntax error on a good day and a silent bug on a bad one.

Combining conditions

and · or · not

```
age = 20
has_ticket = True

age >= 18 and has_ticket    # True  – both
age >= 18 or has_ticket     # True  – at least one
not has_ticket              # False – flips it
```

A	B	A and B	A or B
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

English is looser than Python. "if the mark is 28 or 30" has to be written `mark == 28 or mark == 30` — the second comparison needs its own left-hand side.

Python stops early: in A and B, if A is False it never looks at B. Useful later, and occasionally the reason something does not crash.

if, elif, else

```
if condition:
    do this
elif other_condition:
    do that
else:
    do the fallback
```

```
mark = 24
if mark >= 28:
    print("excellent")
elif mark >= 24:
    print("good")
else:
    print("pass")
```

- The colon ends the question.
- The indented block is what happens if it holds.
- elif is checked only if everything above it failed.
- else catches everything left. It has no condition.
- Exactly one branch runs. Never two.

elif and else are optional. A bare if is a perfectly good program — it just does nothing when the condition is False.

Indentation is the grammar

Not decoration

Inside the if

```
if mark >= 18:  
    print("pass")  
    print("well done")
```

mark = 10 -> nothing at all

Outside the if

```
if mark >= 18:  
    print("pass")  
print("well done")
```

mark = 10 -> well done

Same words, different program. The indentation is the only thing that differs, and it changes what the program means.

- Four spaces per level — your editor (most of them) already does this when you press Tab
- Pasted code can bring in real tabs with it. Tabs and spaces mixed inside one block give a `TabError`, even though the two look identical on screen!

Worked example

A grade classifier, written twice

First attempt — nested

```
if mark >= 18:
    if mark >= 24:
        if mark >= 28:
            print("excellent")
        else:
            print("good")
    else:
        print("pass")
else:
    print("fail")
```

Correct, but hard to read. Every new band adds a level.

Second attempt — elif

```
if mark >= 28:
    print("excellent")
elif mark >= 24:
    print("good")
elif mark >= 18:
    print("pass")
else:
    print("fail")
```

Same behaviour, but flat.
Every band is one line, and they read in order.

When you find yourself indenting a third time, ask whether elif would flatten it.

Order matters

The trap from sheet 1, in Python

Wrong

```
if amount > 50:
    discount = 0.10
elif amount > 100:
    discount = 0.20
else:
    discount = 0
```

amount = 150 gets 10%. The second branch is unreachable: anything over 100 is also over 50.

Right

```
if amount > 100:
    discount = 0.20
elif amount > 50:
    discount = 0.10
else:
    discount = 0
```

Most specific first. Each branch only has to rule out the ones above it.

The wrong version runs, raises nothing, and quietly charges the wrong price.

Three bugs in every beginner conditional

1

= instead of ==

```
if mark = 18:
```

SyntaxError. Python catches this one — be grateful.

2

The band boundary

```
if temp > 30: vs >= 30
```

30 itself falls through. Decide deliberately, every time.

3

The English trap

```
if mark == 28 or 30:
```

Always True. It reads as English and means something else.

Numbers 2 and 3 do not raise anything. They just give the wrong answer — which is worse.

Two conveniences

And one thing not to do

Chained comparisons

```
if 18 <= mark <= 30:
    print("valid mark")

# instead of
if mark >= 18 and mark <= 30:
```

Reads like mathematics, and Python means it.

Comparing floats

```
if 0.1 + 0.2 == 0.3:      # never True

# do this instead
if abs(total - 0.3) < 1e-9:
```

Last week's lesson, now with somewhere to go wrong.

Integers compare exactly, so `==` is safe for them. It is only decimals that need the tolerance.

Worked example

Leap year — your algorithm from week one

```
year = int(input("Year: "))

if year % 4 != 0:
    leap = False
elif year % 100 != 0:
    leap = True
elif year % 400 != 0:
    leap = False
else:
    leap = True

print(f"{year}: leap year = {leap}")
```

- The same four rules you wrote in English in sheet 1, A1.
- Each elif is reached only when everything above it failed — which is exactly how the rules were stated.
- Test it on 1900 (not a leap year) and 2000 (a leap year).

Note what the program does NOT do: it never prints inside the branches. Each branch sets one variable, and the printing happens once, at the end. That habit scales, while printing in every branch does not.

Exercises — Block B

Sheet 03, part B · 40 minutes · plan in comments first

Write

- A ticket price with age bands.
- A password check with two conditions.
- The discount program, ordered correctly.
- A BMI classifier with three bands.

Read and predict

- Six conditions: True or False?
- Four programs: which branch runs?
- Three that are wrong but raise nothing.

Every "write" question starts with the comment plan — including the branches. Writing "# if they are under 6, free" before the code is how you avoid the ordering bug.

Today in one slide

input() always returns text

Wrap it in `int()` or `float()` the moment you need a number.

f-strings

`f"{value:>8.2f}"` — put the value in, line the columns up, two decimals.

A condition is True or False

`==` asks, `=` assigns. Mixing them up is the classic typo.

if / elif / else

Exactly one branch runs, and they are checked in order.

Indentation is the grammar

It decides what is inside the `if`. Four spaces, never tabs.

Most specific first

A branch that can never be reached is a bug that never announces itself.

Before next time

- Finish sheet 03. Solutions go up after we discuss them.
- Re-run your leap year program on 1900, 2000 and 2024 until you believe it.
- Read chapter 1.3 of Sedgewick, Wayne & Dondero.
- Bring the laptop.

Next: Lecture 04 — loops. Today a program could choose. Next week it can repeat, and that is where programming starts to pay.